7. Assume that a process detects and handles stack overflow by protecting its stack region boundary with a *write protected* page. Hence, any write to this page due to stack overflow will result in a *SIGSEGV* being generated. Since the process stack has already overflowed, how can the signal handler for the *SIGSEGV* be executed?
8. If a process is multi-threaded and one of the threads invokes a *fork* system call, what will happen? What will happen if one of the threads executes an *exec* system call?
9. Assume that you are asked to write code for a uni-processor system, in which the instruction set of the underlying microprocessor does not support any form of CAS (Compare And Swap), CMPXCHG or Load-Linked/Store-Conditional instructions. However, you are asked to implement a basic primitive locking mechanism in assembly routine for that processor. Is it possible for you to implement the same? Would your answer change if you were asked to do this for a multi-processor system with the same constraints?
10. Assume that you are given an operating system in which there is a condition that a process cannot try to acquire any additional lock while already holding one lock. Is it possible for a deadlock condition to arise in such a system?
11. You are given a multi-processor system in which the cache line granularity at the hardware level is 128 bytes. Basically, each cache line size is 128 bytes. Now you have an operating system that you are porting on to that hardware, which assumes that the cache line granularity is 64 bytes. You are not allowed to change the cache line size in operating system code to 128 bytes. What performance or correctness issues would you run into by having to obey this constraint? Is it possible for you to work around this issue in your application code, though you can't make changes to the operating system code? A slight variant of the above is the case where the hardware supports a 64 byte cache line size, whereas the OS assumes a cache line size of 128 bytes, which you are not allowed to change. What would be the issues in that case?
12. Your friend in the operating system course tells you that the more fine-grained locking granularity you use in your application code, the greater are the chances of deadlock. Is he correct? If not, how can you work around this issue? *Hint:* Ask him if he knows about transactional memory.
13. Can you differentiate between *write-through*, *write-allocate* and *write-back* caches in modern processors? What kind of caches does Intel's latest Haswell processor employ?
14. Why is it that in computer systems, the first level of cache closest to the processor is typically direct-mapped whereas further levels of cache hierarchy are typically set-associative? What would be the issue if the first level of cache is fully-associative instead of being direct-mapped?
15. Assume that you know the memory footprint of the application you are planning to run. Your computer architect has designed the system that has cache memory equal to 2X of the memory footprint of your application. Also assume that other than OS system processes, your application is the only one that will be executing on the system. Can you say for sure that your application will not suffer any cache misses? If not, why not?
16. Today's file systems are designed for rotational hard disk latencies. For instance, many modern file systems use data structures like B-trees to compensate for the rotational disk delays in bringing data from the disk. However, with the advent of solid state disks (SSD) which have no rotational components, most modern storage systems are moving away from rotational HDDs to SSDs as their underlying storage media. If you are asked to design a file system for a modern storage system consisting purely of solid state drives, how would the file system design differ from those traditional file systems? What kind of pitfalls would you encounter in an SSD-based storage system?
17. What would happen if you have configured your system with extremely low swap space? How would you determine that you are suffering from low swap space on a Linux system?
18. Can you explain what is meant by cache coherence in a multi-processor system? Is it possible to design a multi-processor system without any hardware support for cache coherence and still achieve correctness by making changes in the operating system and applications?

My 'must-read book' for this month

This month's must-read book suggestion comes from one of our readers, Radha Premkumar. The book is, *'Refactoring: Improving the Design of Existing Code'* by Martin Fowler. Radha says, "This book is an important guide to writing quality code that can be maintained well by the proper use of refactoring techniques. It helps to improve existing code as well as improve the design of new code." Thanks Radha, for your suggestion.

If you have a favourite programming book/article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful so I can mention it in this column. This would help many readers who want to improve their software skills.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Till we meet again next month, happy programming and here's wishing you the very best! END

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working with Hewlett Packard India Ltd. Her interests include compilers, multi-core and storage systems. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at *http://www.linkedin.com/groups?home=HYPERLINK "http://www.linkedin.com/groups?home=&gid=2339182"&HYPERLINK "http://www.linkedin.com/groups?home=&gid=2339182"gid=2339182*